

CURSO DE ENGENHARIA DE SOFTWARE

DAS - DESIGN E ARQUITETURA DE SISTEMAS

ARQUITETURA EM CAMADAS COM JAVA E SPRING BOOT

Material didático prático, incremental e orientado a entregas

Professor: Prof. Sestito

Joinville - SC

2026

APRESENTAÇÃO

Este material orienta o desenvolvimento incremental de uma API para um Sistema de Chamados, utilizando Java 21, Spring Boot, Maven, Spring Web, Spring Data JPA, H2, Bean Validation e Thymeleaf. O foco da disciplina é compreender as decisões de design e arquitetura, não apenas fazer o código funcionar.

Regra acadêmica obrigatória: *O uso de ferramentas de Inteligência Artificial nesta etapa é proibido. O objetivo é consolidar raciocínio, prática de programação, leitura de erros, arquitetura em camadas e domínio do fluxo de desenvolvimento. Em etapas futuras da disciplina, ferramentas de IA poderão ser utilizadas de forma orientada.*

Validação antes de commit: *Antes de qualquer commit de entrega, o aluno deve chamar o professor para validar a etapa. Somente após a validação o commit poderá ser realizado. Esta validação é obrigatória.*

Dúvidas: *Em caso de dificuldade técnica, erro de execução, dúvida de arquitetura ou dúvida sobre a entrega, chame o professor antes de alterar o fluxo proposto.*

1 OBJETIVOS DE APRENDIZAGEM

- Compreender a separação de responsabilidades em uma arquitetura em camadas.
- Construir uma API REST aplicando Controller, Service, Repository, Entity, DTO e tratamento de exceções.
- Entender o caminho completo de uma requisição HTTP até o banco de dados e seu retorno.
- Praticar versionamento com Git e GitHub por meio de branches, commits, push e Pull Request.
- Validar rotas por terminal, Insomnia ou Postman e interpretar códigos HTTP.
- Representar o sistema por diagramas de caso de uso, sequência, classes e entidade-relacionamento.
- Evoluir o projeto por entregas pequenas, testáveis e revisáveis.

2 ORGANIZAÇÃO DAS AULAS E ENTREGAS

Cada aula deve produzir, no mínimo, uma entrega verificável. A recomendação é trabalhar em pequenos incrementos, mantendo o projeto sempre executável. As etapas abaixo podem ser distribuídas conforme o ritmo da turma.

Aula/etapa	Objetivo mínimo	Entrega
1	Gerar projeto, executar localmente e publicar no GitHub	Repositório inicial funcional + branch criada
2	Organizar arquitetura em camadas e criar entidade Cliente	Estrutura de pacotes + Cliente.java
3	DTOs, Repository e Service	Fluxo de dados até a camada de serviço
4	Controller + POST e GET	Rotas testadas e evidências
5	PUT, DELETE e exceções	CRUD completo com tratamento de erros
6	Categoria	CRUD de Categoria
7	Técnico/Usuário	CRUD de Técnico
8	Chamado + enums + relacionamentos	Núcleo do domínio implementado
9	Regras de status e atribuição	PATCH e regras de negócio
10	Filtros, paginação e comentários	Consultas e interação
11	Testes automatizados	Repository e Controller testados
12	Documentação e banco real	OpenAPI + profile PostgreSQL + PR final

3 ETAPA 1 - CRIAÇÃO DO PROJETO NO SPRING INITIALIZR

A primeira tela define a base técnica do projeto. Cada opção selecionada influencia o arquivo pom.xml, a versão do Java, o empacotamento e as bibliotecas disponíveis no código.

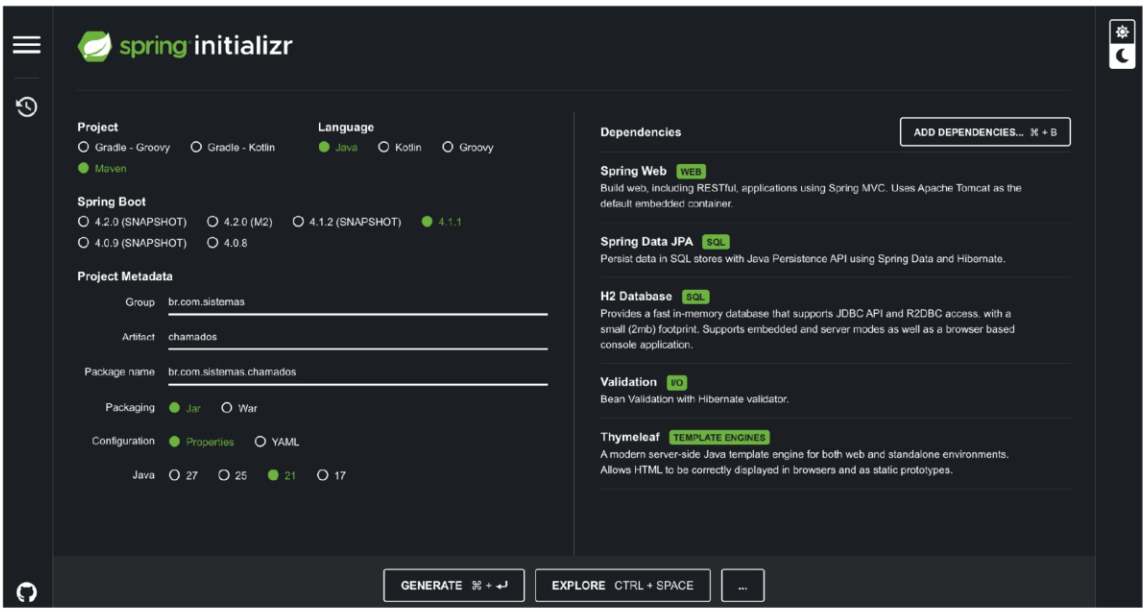


Figura 1 - Configuração do projeto no Spring Initializr
Fonte: elaboração própria / material fornecido pelo professor.

Campo	Valor recomendado	O que significa
Project	Maven	Gerenciador de dependências e ciclo de build.
Language	Java	Linguagem utilizada na disciplina.
Spring Boot	4.1.1 (estável exibida na tela)	Versão da plataforma Spring Boot usada para iniciar o projeto.
Group	br.com.sistemas	Identificador base/namespace do projeto.
Artifact	chamados	Nome do artefato e diretório gerado.
Package name	br.com.sistemas.chamados	Pacote raiz; todas as camadas devem ficar abaixo dele.
Packaging	Jar	Empacotamento padrão para aplicação Spring Boot.
Java	21	Versão LTS utilizada no projeto.

Dependências iniciais: Spring Web, Spring Data JPA, H2 Database, Validation e Thymeleaf. Neste primeiro módulo o Thymeleaf ficará disponível para evolução futura; o foco imediato será a API REST.

Não usar SNAPSHOT: Selecione uma versão estável do Spring Boot. Versões SNAPSHOT ou milestone podem mudar comportamento e não são recomendadas para esta atividade.

4 ETAPA 2 - ABRIR, EXECUTAR E ENTENDER O PRIMEIRO RESULTADO

Descompacte o projeto e abra a pasta raiz no VS Code. O uso do VS Code é recomendado para padronizar a aula, mas o IntelliJ IDEA pode ser utilizado. O terminal deve estar aberto na pasta que contém pom.xml e mvnw.

macOS / Linux

```
./mvnw spring-boot:run
```

Windows PowerShell

```
.\mvnw spring-boot:run
```

Quando o terminal indicar que a aplicação iniciou e está ouvindo a porta 8080, abra o navegador em http://localhost:8080/. Neste momento é esperado que apareça uma página de erro 404.

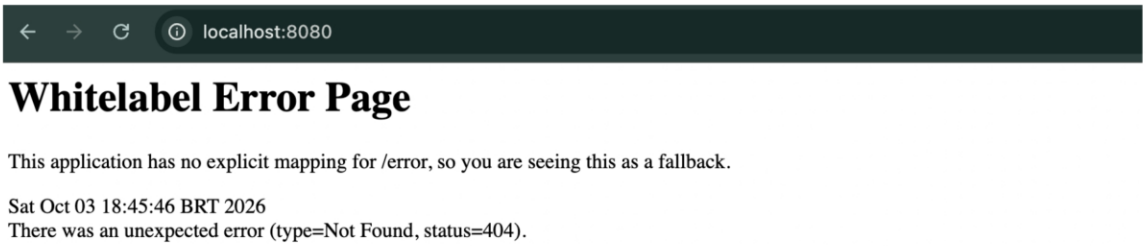


Figura 2 - Whitelabel Error Page ao acessar a raiz /

Fonte: elaboração própria / material fornecido pelo professor.

O 404 não significa que o projeto falhou: A aplicação está rodando. O status 404 aparece porque ainda não existe um Controller com rota mapeada para /. Depois criaremos as rotas em /clientes. A evidência de que o servidor está ativo é justamente a resposta HTTP recebida do Spring Boot.

5 ETAPA 3 - VERSIONAMENTO ANTES DO DESENVOLVIMENTO DE CLIENTE

Antes de iniciar o fluxo de Cliente, o projeto deve estar versionado. Esta etapa faz parte da avaliação do processo de engenharia de software.



Figura 3 - Fluxo obrigatório de versionamento e revisão

Fonte: elaboração própria / material fornecido pelo professor.

No GitHub, crie um novo repositório para o projeto e adicione o usuário Profesestito como membro/colaborador. Em seguida, no terminal do projeto:

Preparação do repositório local

```
git init
git add .
git status
```

PARE AQUI: Antes do primeiro commit, chame o professor para validar o projeto, o pom.xml, a execução e o status do Git. O commit só deve ser feito após a validação.

Após validação do professor

```
git commit -m "chore: inicializa projeto Spring Boot"
git branch -M main
git remote add origin URL_DO_REPOSITORIO
git push -u origin main
```

Criar a branch de desenvolvimento

```
git checkout -b feature/nome_aluno-developer
```

Substitua ***nome_aluno*** pelo seu nome, mantendo o padrão solicitado. ***Exemplo***: **feature/joao-silva-developer**. Todo o desenvolvimento das etapas seguintes deve ocorrer nessa branch, salvo orientação diferente do professor.

6 ETAPA 4 - ORGANIZAÇÃO DA ARQUITETURA EM CAMADAS

Dentro de src/main/java/br/com/sistemas/chamados, crie os pacotes controller, dto, entity, exception, repository e service. A classe ChamadosApplication.java permanece no pacote raiz para que o Spring faça o *component scan* das camadas abaixo dela.

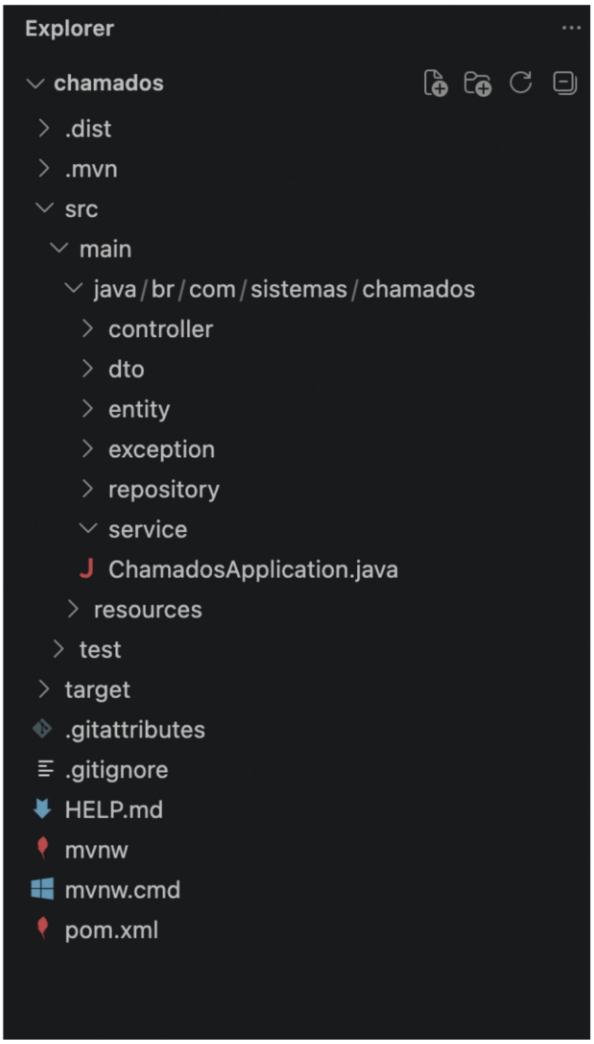


Figura 4 - Estrutura inicial de pacotes em camadas no VS Code

Fonte: elaboração própria / material fornecido pelo professor.

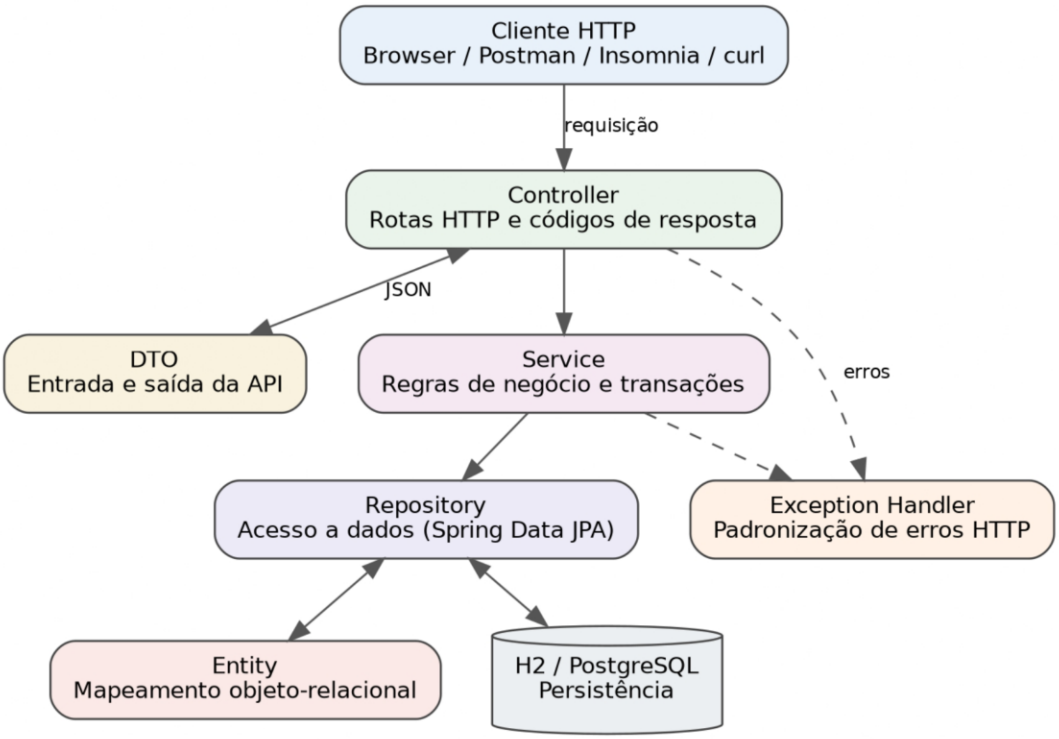


Figura 5 - Responsabilidades e fluxo da arquitetura em camadas

Fonte: elaboração própria / material fornecido pelo professor.

Camada	Responsabilidade	Não deve fazer
Controller	Receber HTTP, ler parâmetros/JSON, chamar Service e devolver status/resposta.	Conter regra de negócio ou SQL.
DTO	Definir contrato de entrada/saída da API.	Representar diretamente a tabela do banco.
Service	Aplicar regras de negócio e coordenar transações.	Conhecer detalhes de HTTP.
Repository	Realizar acesso a dados via Spring Data JPA.	Implementar regra de negócio.
Entity	Representar o modelo persistido no banco.	Controlar respostas HTTP.
Exception	Representar falhas e convertê-las em respostas HTTP coerentes.	Misturar regras de persistência.

7 ETAPA 5 - CONFIGURAÇÃO DO H2 E DA APLICAÇÃO

No arquivo src/main/resources/application.properties, adicione a configuração abaixo. Ela cria um banco H2 em memória para fins didáticos e habilita o console web do banco.

src/main/resources/application.properties

```
spring.application.name=chamados

spring.datasource.url=jdbc:h2:mem:chamadosdb
```

```
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

O banco em memória é recriado sempre que a aplicação é reiniciada. Isso é útil em aula porque reduz a configuração externa. Mais adiante, o projeto poderá usar PostgreSQL por profile.

8 ETAPA 6 - IMPLEMENTAÇÃO DO FLUXO DE CLIENTE

A implementação será feita de dentro para fora:

Entity → Repository → DTOs → Exceptions → Service → Controller.

Dessa forma, cada camada depende apenas da camada imediatamente necessária e o aluno consegue compreender o fluxo completo.

8.1 Entity - Cliente.java

A Entity é a representação orientada a objetos da tabela clientes. As anotações JPA informam como a classe será persistida. ***@Id*** define a chave primária; ***@GeneratedValue*** pede geração automática; ***@Column*** define obrigatoriedade, tamanho e unicidade.

src/main/java/br/com/sistemas/chamados/entity/Cliente.java

```
package br.com.sistemas.chamados.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

/** ENTITY: representa a tabela do banco. Cada objeto = uma linha. */
@Entity
@Table(name = "clientes")
public class Cliente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @Column(nullable = false, length = 100)
    private String nome;

    @Column(nullable = false,  unique = true, length = 120)
    private String email;

    @Column(length = 20)
    private String telefone;

    public Cliente() { }

    public Cliente(String nome, String email, String telefone) {
        this.nome = nome;
        this.email = email;
        this.telefone = telefone;
    }

    public Long getId() { return id; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getTelefone() { return telefone; }
    public void setTelefone(String telefone) { this.telefone = telefone; }
}
```

Checkpoint de compreensão: A propriedade `unique = true` indica que o valor armazenado nesta coluna deve ser único no banco de dados, ou seja, não podem existir dois clientes cadastrados com o mesmo e-mail.

8.2 Repository - ClienteRepository.java

O Repository abstrai a persistência. Ao estender JpaRepository<Cliente, Long>, ganhamos operações como save, findById, findAll e delete. Os métodos existsBy... demonstram query derivada por nome: o Spring interpreta o nome e gera a consulta.

repository/ClienteRepository.java

```
package br.com.sistemas.chamados.repository;

import br.com.sistemas.chamados.entity.Cliente;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ClienteRepository extends JpaRepository<Cliente, Long> {
    boolean existsByEmail(String email);
    boolean existsByEmailAndIdNot(String email, Long id);
}
```

8.3 DTOs - contrato da API

ClienteRequest representa o JSON que entra; *ClienteResponse* representa o JSON que sai. Separar DTO de Entity evita expor a persistência diretamente e permite alterar contratos sem alterar a tabela.

dto/ClienteRequest.java

```
package br.com.sistemas.chamados.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record ClienteRequest(
    @NotBlank(message = "Nome é obrigatório")
    @Size(max = 100, message = "Nome deve ter no máximo 100 caracteres")
    String nome,

    @NotBlank(message = "E-mail é obrigatório")
    @Email(message = "E-mail inválido")
    String email,

    @Size(max = 20, message = "Telefone deve ter no máximo 20 caracteres")
    String telefone
) { }
```

dto/ClienteResponse.java

```
package br.com.sistemas.chamados.dto;

import br.com.sistemas.chamados.entity.Cliente;

public record ClienteResponse(Long id, String nome, String email, String telefone) {
    public static ClienteResponse de(Cliente c) {
        return new ClienteResponse(c.getId(), c.getNome(), c.getEmail(), c.getTelefone());
    }
}
```

8.4 Exceptions - erros previsíveis e respostas HTTP

Exceções de domínio deixam o Service legível e permitem que o Controller permaneça sem blocos repetidos de try/catch. O ***@RestControllerAdvice*** intercepta exceções de todos os controllers e monta uma resposta padronizada.

Correção importante: No material-base, o conteúdo de RecursoNaoEncontradoException.java estava repetindo a classe RegraNegocioException. A versão abaixo corrige essa inconsistência para que a busca por ID inexistente resulte em HTTP 404.

exception/RecursoNaoEncontradoException.java

```
package br.com.sistemas.chamados.exception;

/** Lançada quando um recurso procurado não existe (vira HTTP 404). */
public class RecursoNaoEncontradoException extends RuntimeException {
    public RecursoNaoEncontradoException(String mensagem) {
        super(mensagem);
    }
}
```

exception/RegraNegocioException.java

```
package br.com.sistemas.chamados.exception;

/** Lançada quando uma regra de negócio é violada (vira HTTP 409). */
public class RegraNegocioException extends RuntimeException {
    public RegraNegocioException(String mensagem) {
        super(mensagem);
    }
}
```

exception/ErroResposta.java

```
package br.com.sistemas.chamados.exception;

import java.time.LocalDateTime;
import java.util.List;

public record ErroResposta(
    LocalDateTime momento,
    int status,
    String erro,
    List<String> detalhes
) { }
```

exception/GlobalExceptionHandler.java

```
package br.com.sistemas.chamados.exception;

import java.time.LocalDateTime;
import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RecursoNaoEncontradoException.class)
    public ResponseEntity<ErroResposta> naoEncontrado(RecursoNaoEncontradoException ex) {
        return montar(HttpStatus.NOT_FOUND, ex.getMessage(), List.of());
    }
```

```
    @ExceptionHandler(RegraNegocioException.class)
    public ResponseEntity<ErroResposta> regraNegocio(RegraNegocioException ex) {
        return montar(HttpStatus.CONFLICT, ex.getMessage(), List.of());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErroResposta> validacao(MethodArgumentNotValidException ex) {
        List<String> detalhes = ex.getBindingResult().getFieldErrors().stream()
            .map(e -> e.getField() + ": " + e.getDefaultMessage())
            .toList();
        return montar(HttpStatus.BAD_REQUEST, "Dados inválidos", detalhes);
    }

    private ResponseEntity<ErroResposta> montar(
        HttpStatus status, String erro, List<String> detalhes) {
        return ResponseEntity.status(status)
            .body(new ErroResposta(LocalDateTime.now(), status.value(), erro, detalhes));
    }
}
```

8.5 Service - ClienteService.java

O Service concentra regras de negócio. O Controller não pergunta se o e-mail já existe; essa responsabilidade é do Service. A anotação ***@Transactional*** define a fronteira transacional. Nas consultas, ***readOnly = true*** sinaliza operação apenas de leitura.

service/ClienteService.java

```
package br.com.sistemas.chamados.service;

import java.util.List;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import br.com.sistemas.chamados.dto.ClienteRequest;
import br.com.sistemas.chamados.dto.ClienteResponse;
import br.com.sistemas.chamados.entity.Cliente;
import br.com.sistemas.chamados.exception.RecursoNaoEncontradoException;
import br.com.sistemas.chamados.exception.RegraNegocioException;
import br.com.sistemas.chamados.repository.ClienteRepository;

@Service
public class ClienteService {

    private final ClienteRepository repository;

    public ClienteService(ClienteRepository repository) {
        this.repository = repository;
    }

    @Transactional
    public ClienteResponse criar(ClienteRequest dto) {
        if (repository.existsByEmail(dto.email())) {
            throw new RegraNegocioException("Já existe cliente com o e-mail " + dto.email());
```

```
        }
        Cliente salvo = repository.save(new Cliente(dto.nome(), dto.email(), dto.telefone()));
        return ClienteResponse.de(salvo);
    }

    @Transactional(readOnly = true)
    public List<ClienteResponse> listar() {
        return repository.findAll().stream().map(ClienteResponse::de).toList();
    }

    @Transactional(readOnly = true)
    public ClienteResponse buscar(Long id) {
        return ClienteResponse.de(buscarEntity(id));
    }

    @Transactional
    public ClienteResponse atualizar(Long id, ClienteRequest dto) {
        Cliente cliente = buscarEntity(id);
        if (repository.existsByEmailAndIdNot(dto.email(), id)) {
            throw new RegraNegocioException("Já existe cliente com o e-mail " + dto.email());
        }
        cliente.setNome(dto.nome());
        cliente.setEmail(dto.email());
        cliente.setTelefone(dto.telefone());
        return ClienteResponse.de(cliente);
    }

    @Transactional
    public void excluir(Long id) {
        repository.delete(buscarEntity(id));
    }

    private Cliente buscarEntity(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new RecursoNaoEncontradoException(
                "Cliente " + id + " não encontrado"));
    }
}
```

8.6 Controller - ClienteController.java

O Controller é a porta de entrada HTTP. ***@RequestMapping("/clientes")*** define a base. Cada método adiciona o verbo HTTP e, quando necessário, o ID. @Valid dispara as validações do ***ClienteRequest*** antes de chamar o Service.

controller/ClienteController.java

```
package br.com.sistemas.chamados.controller;

import java.net.URI;
import java.util.List;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
```

```
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import br.com.sistemas.chamados.dto.ClienteRequest;
import br.com.sistemas.chamados.dto.ClienteResponse;
import br.com.sistemas.chamados.service.ClienteService;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/clientes")
public class ClienteController {

    private final ClienteService service;

    public ClienteController(ClienteService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<ClienteResponse> criar(
        @Valid @RequestBody ClienteRequest dto) {
        ClienteResponse criado = service.criar(dto);
        return ResponseEntity.created(URI.create("/clientes/" + criado.id())).body(criado);
    }

    @GetMapping
    public List<ClienteResponse> listar() {
        return service.listar();
    }

    @GetMapping("/{id}")
    public ClienteResponse buscar(@PathVariable Long id) {
        return service.buscar(id);
    }

    @PutMapping("/{id}")
    public ClienteResponse atualizar(
        @PathVariable Long id,
        @Valid @RequestBody ClienteRequest dto) {
        return service.atualizar(id, dto);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> excluir(@PathVariable Long id) {
        service.excluir(id);
        return ResponseEntity.noContent().build();
    }
}
```

8.7 Classe principal - ChamadosApplication.java

A classe principal inicia o Spring Boot. Como está no pacote raiz br.com.sistemas.chamados, o Spring consegue localizar automaticamente os componentes das subpastas controller, service e demais pacotes.

ChamadosApplication.java

```
package br.com.sistemas.chamados;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ChamadosApplication {
    public static void main(String[] args) {
        SpringApplication.run(ChamadosApplication.class, args);
    }
}
```

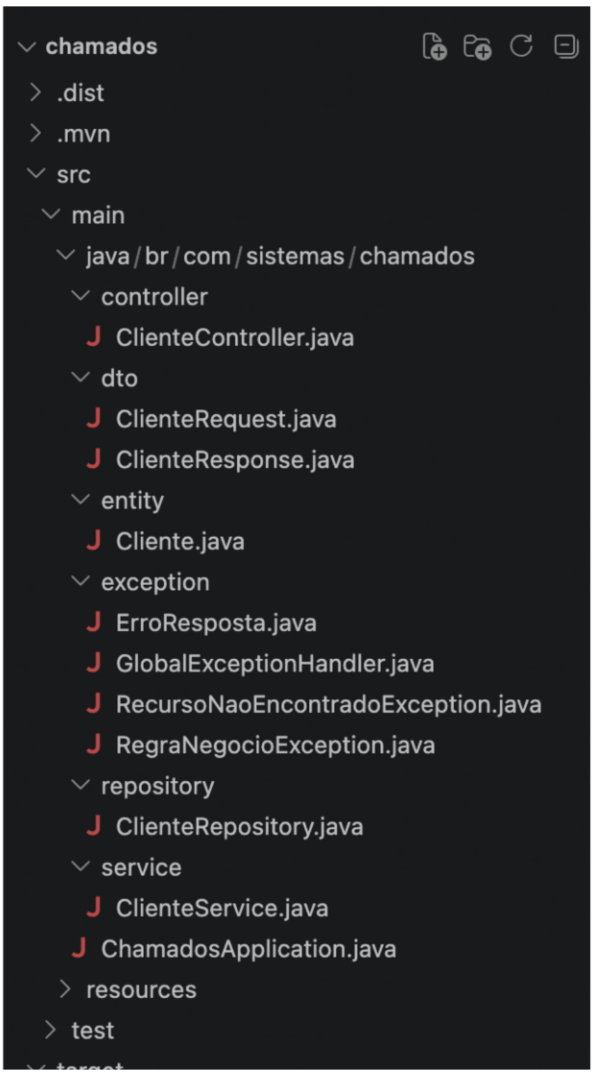


Figura 6 - Estrutura esperada após concluir o fluxo de Cliente

Fonte: elaboração própria / material fornecido pelo professor.

9 MODELAGEM E DIAGRAMAS DO SISTEMA

Os diagramas abaixo representam visões complementares. Nenhum diagrama substitui outro: caso de uso mostra objetivos dos atores; sequência mostra interação temporal; classes mostra estrutura do software; DER mostra estrutura dos dados; mapa de telas antecipa o front-end.

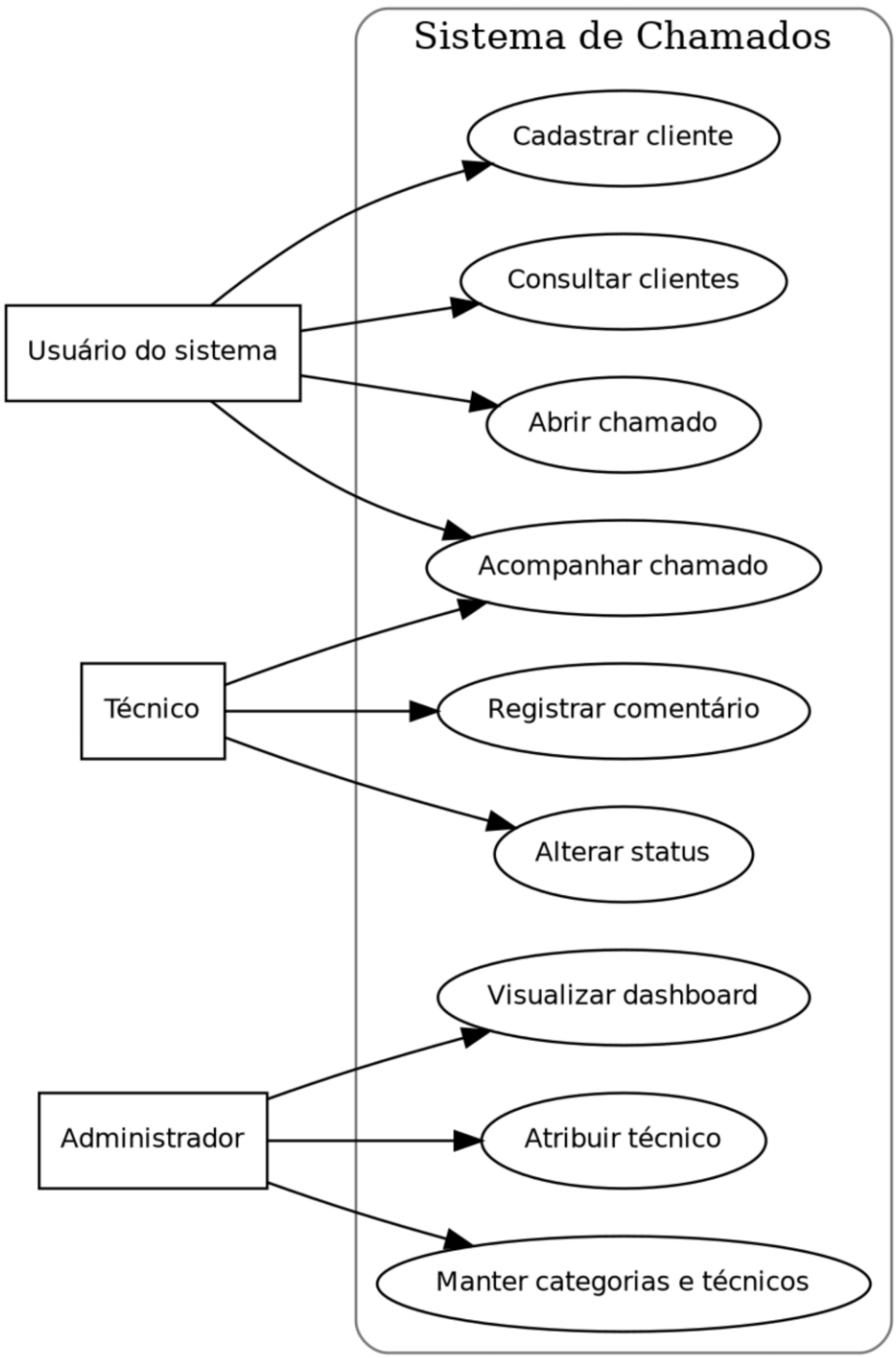


Figura 7 - Diagrama de caso de uso - visão funcional

Fonte: elaboração própria / material fornecido pelo professor.

No primeiro incremento, apenas o caso de uso de manutenção de clientes está implementado. Os demais orientam a evolução das próximas aulas.



Figura 8 - Diagrama de sequência - criação de cliente

Fonte: elaboração própria / material fornecido pelo professor.

Observe que o cliente HTTP nunca acessa o Repository diretamente. O Controller delega ao ***Service***, o Service aplica regras e usa o Repository, que persiste no banco.

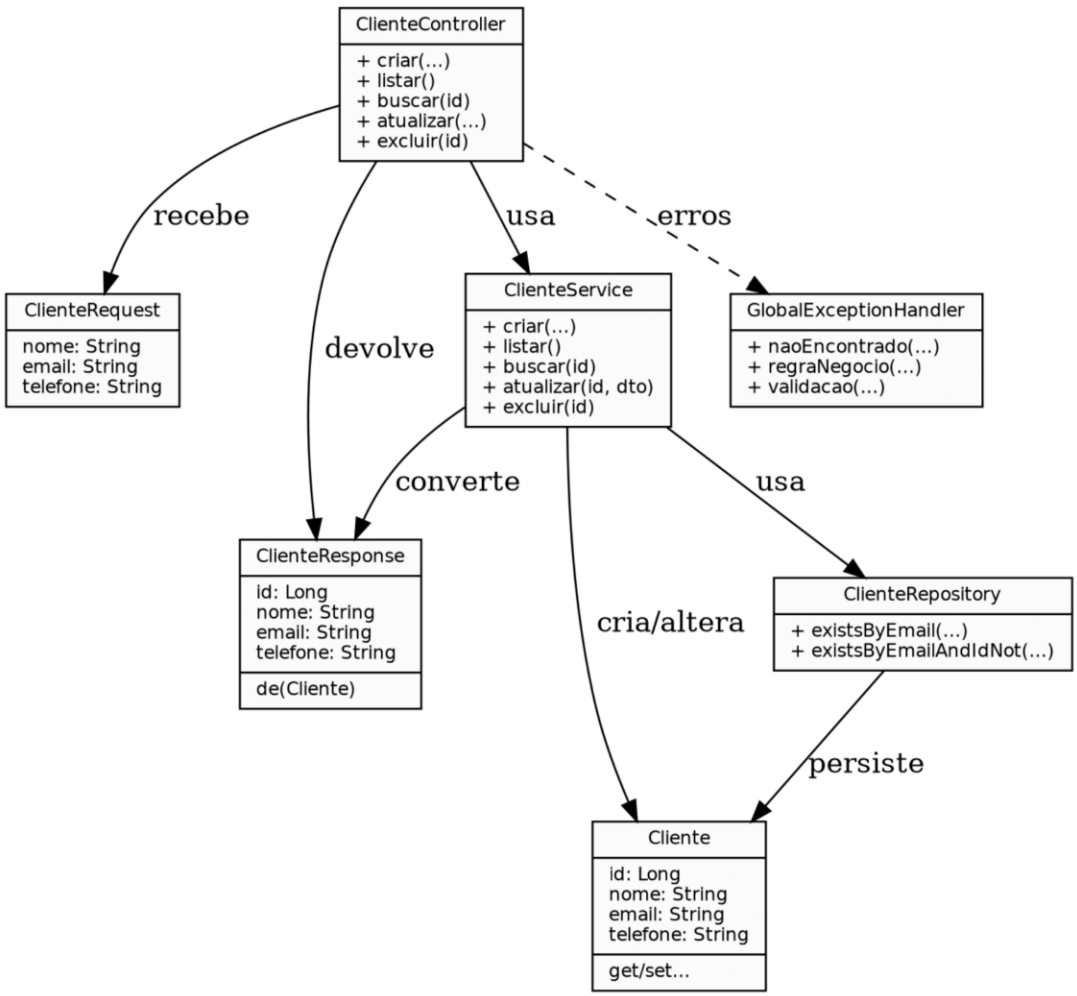


Figura 9 - Diagrama de classes - fluxo de Cliente

Fonte: elaboração própria / material fornecido pelo professor.

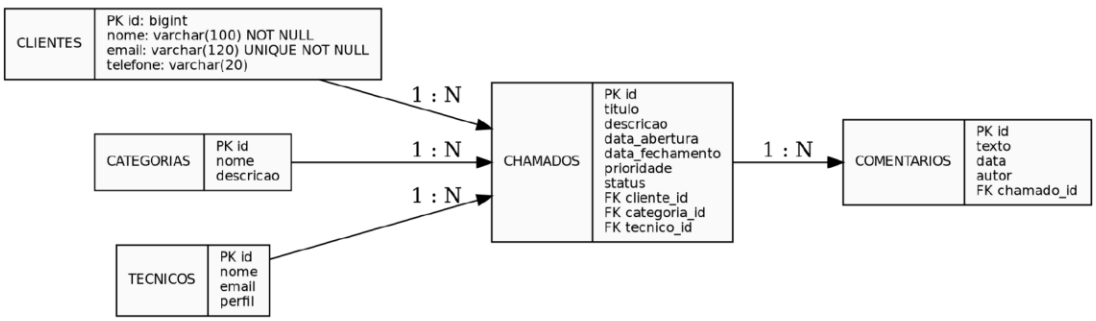


Figura 10 - DER planejado do Sistema de Chamados

Fonte: elaboração própria / material fornecido pelo professor.

No momento em que somente Cliente está implementado, apenas a tabela CLIENTES existe. CATEGORIAS, TECNICOS, CHAMADOS e COMENTARIOS serão criados nas próximas entregas.

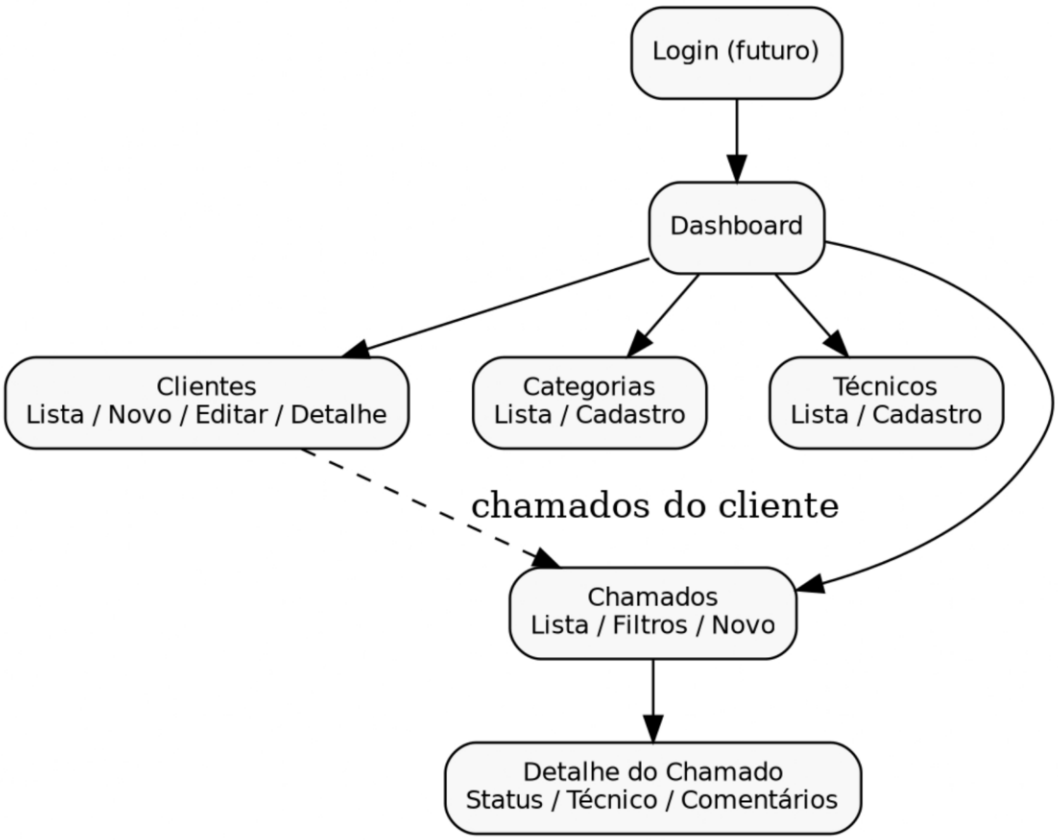


Figura 11 - Mapa de telas planejado para o front-end

Fonte: elaboração própria / material fornecido pelo professor.

O front-end ainda não será implementado. O mapa de telas serve para conectar o domínio da API às futuras interfaces e ajudar a turma a visualizar quais recursos cada endpoint atenderá.

10 TESTE DAS ROTAS - TERMINAL, INSOMNIA E POSTMAN

Sempre execute a aplicação antes dos testes. Para cada rota, verifique três elementos: método HTTP, URL e corpo JSON quando houver. Além da resposta, observe o status HTTP.

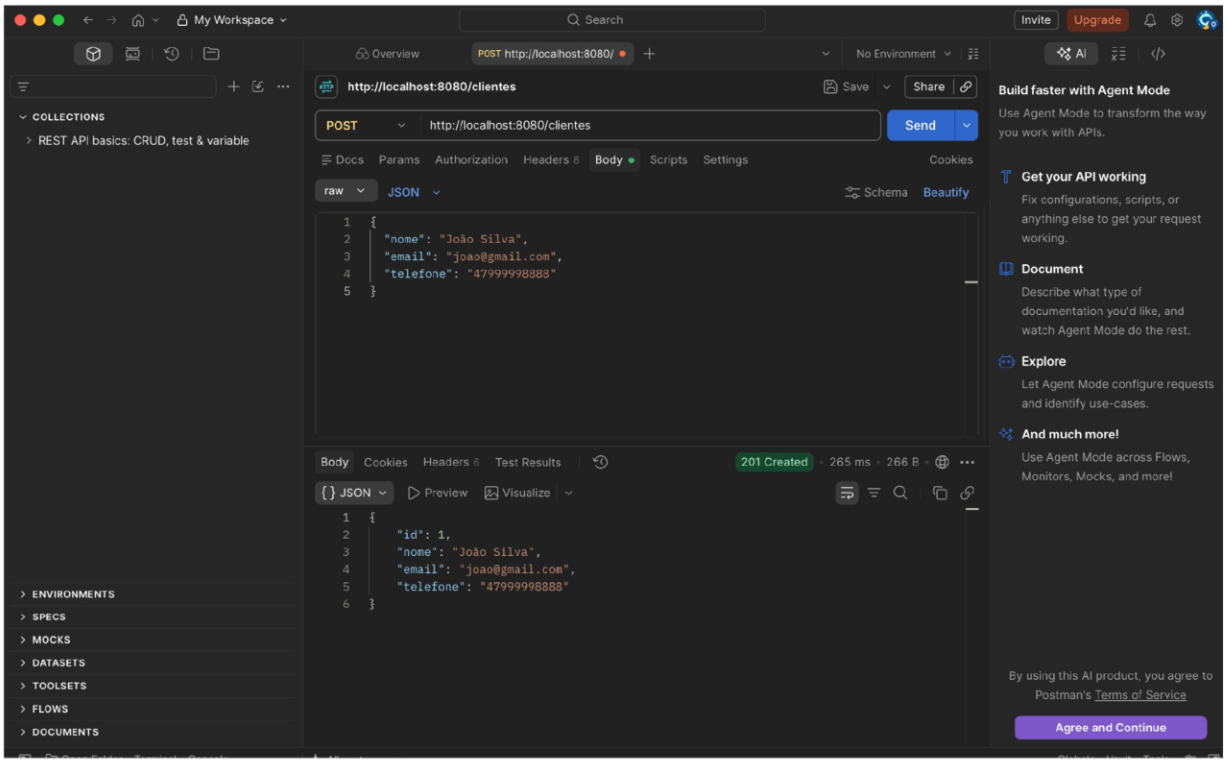
Método	Rota	Ação	Sucesso esperado
POST	/clientes	Criar cliente	201 Created
GET	/clientes	Listar todos	200 OK
GET	/clientes/{id}	Buscar por ID	200 OK
PUT	/clientes/{id}	Atualizar	200 OK
DELETE	/clientes/{id}	Excluir	204 No Content

10.1 POST /clientes - criar cliente

Teste pelo terminal (macOS/Linux; no Windows use curl.exe se necessário)

```
curl -i -X POST http://localhost:8080/clientes \
  -H "Content-Type: application/json" \
  -d '{"nome":"Ana Souza","email":"ana@exemplo.com","telefone":"47999999999"}'
```

Teste pelo Postman



Siga estas etapas:

1. Clique no botão +, à esquerda, e selecione HTTP.

2. Na nova aba, selecione o método `POST`.
3. Informe a URL `http://localhost:8080/clientes`.
4. Entre na aba Body.
5. Selecione raw → JSON (no Postman). No Insomnia, selecione Body → JSON.
6. Insira o conteúdo abaixo:

```
{
  "nome": "João Silva",
  "email": "joao@gmail.com",
  "telefone": "47999998888"
}
```

Clique em ***Send***.

Resultado esperado

201 Created

10.2 GET /clientes - listar

Teste pelo terminal (macOS/Linux; no Windows use curl.exe se necessário)

```
curl -i http://localhost:8080/clientes
```

No Insomnia/Postman: método GET, URL http://localhost:8080/clientes, sem body. Confirme 200 e um array JSON.

10.3 GET /clientes/{id} - buscar por ID

Teste pelo terminal (macOS/Linux; no Windows use curl.exe se necessário)

```
curl -i http://localhost:8080/clientes/1
```

No Insomnia/Postman: método GET e URL http://localhost:8080/clientes/1. Se o ID existir, espere 200. Teste também um ID inexistente para confirmar 404.

10.4 PUT /clientes/{id} - atualizar

Teste pelo terminal (macOS/Linux; no Windows use curl.exe se necessário)

```
curl -i -X PUT http://localhost:8080/clientes/1 \
  -H "Content-Type: application/json" \
  -d '{"nome":"Ana Souza Atualizada","email":"ana@exemplo.com","telefone":"47988888888"}'
```

No Insomnia/Postman: PUT, URL http://localhost:8080/clientes/1, Body/JSON com os três campos. Confirme 200 e dados atualizados.

10.5 DELETE /clientes/{id} - excluir

Teste pelo terminal (macOS/Linux; no Windows use curl.exe se necessário)

```
curl -i -X DELETE http://localhost:8080/clientes/1
```

No Insomnia/Postman: DELETE, URL http://localhost:8080/clientes/1, sem body. Confirme 204 e depois faça GET /clientes/1 para validar 404.

10.6 Testes obrigatórios de erro

Validação - esperado 400 Bad Request

```
curl -i -X POST http://localhost:8080/clientes \
  -H "Content-Type: application/json" \
  -d '{"nome":"","email":"email-invalido","telefone":"123"}'
```

E-mail duplicado - esperado 409 Conflict

```
curl -i -X POST http://localhost:8080/clientes \
  -H "Content-Type: application/json" \
  -d '{"nome":"Outro Cliente","email":"ana@exemplo.com","telefone":"47977777777"}'
```

Recurso inexistente - esperado 404 Not Found

```
curl -i http://localhost:8080/clientes/99999
```

11 INSPEÇÃO DO BANCO PELO H2 CONSOLE

Acesse http://localhost:8080/h2-console. Use JDBC URL jdbc:h2:mem:chamadosdb, usuário ***sa*** e senha em branco. Depois de conectar, execute SELECT * FROM CLIENTES; para observar os registros persistidos pela API.

Importante: Se a aplicação for reiniciada, o banco H2 em memória perde os dados. Isso é comportamento esperado nesta fase.

12 CHECKPOINT OBRIGATÓRIO - CONCLUSÃO DO FLUXO CLIENTE

Antes de *commitar*, execute todos os testes, deixe a aplicação funcionando e chame o professor. O professor deve validar código, arquitetura, respostas HTTP e organização do Git.

- ☐ Aplicação inicia sem erro.
- ☐ POST retorna 201.
- ☐ GET lista e busca por ID.
- ☐ PUT altera o registro.
- ☐ DELETE retorna 204.
- ☐ ID inexistente retorna 404.
- ☐ E-mail duplicado retorna 409.
- ☐ Dados inválidos retornam 400.
- ☐ H2 mostra a tabela CLIENTES.
- ☐ Professor validou a etapa antes do commit.

Somente após validação do professor

```
git status
git add .
git commit -m "feat: implementa CRUD de clientes em arquitetura em camadas"
git push -u origin feature/nome_aluno-developer
```

No GitHub, abra um Pull Request da branch feature/nome_aluno-developer para main e solicite revisão do Prof Sestito/Profesestito. Não faça merge sem a revisão solicitada.

13 EVOLUÇÃO DO PROJETO - CHECKLISTS POR ETAPA

A partir daqui, o material deixa de fornecer o código completo. O objetivo é que o aluno reaplique o padrão aprendido no fluxo Cliente. Cada novo recurso deve seguir a

ordem Entity → Repository → DTO → Exception (quando necessária) → Service → Controller → Testes → Validação do professor → Commit.

13.1 Categoria

- ☐ Criar entity Categoria com id, nome e descrição.
- ☐ Definir validações e DTOs de entrada/saída.
- ☐ Criar CategoriaRepository.
- ☐ Criar CategoriaService com regra para evitar nomes duplicados.
- ☐ Criar CRUD REST em /categorias.
- ☐ Testar POST, GET, GET/{id}, PUT e DELETE.
- ☐ Validar com o professor antes do commit.
- ☐ Commit sugerido: feat: implementa CRUD de categorias.

13.2 Técnico/Usuário

- ☐ Criar entity Tecnico com id, nome, email e perfil.
- ☐ Criar enum Perfil com ADMIN e TECNICO.
- ☐ Garantir e-mail único.
- ☐ Criar DTOs, Repository, Service e Controller.
- ☐ Disponibilizar CRUD em /tecnicos.
- ☐ Testar casos de sucesso e duplicidade.
- ☐ Validar com o professor antes do commit.
- ☐ Commit sugerido: feat: implementa cadastro de tecnicos.

13.3 Chamado - núcleo do sistema

- ☐ Criar entity Chamado com título, descrição, dataAbertura, dataFechamento, prioridade e status.
- ☐ Criar enums Prioridade: BAIXA, MEDIA, ALTA, CRITICA.
- ☐ Criar enum StatusChamado: ABERTO, EM_ATENDIMENTO, AGUARDANDO_CLIENTE, FECHADO.
- ☐ Mapear @ManyToOne para Cliente, Categoria e Técnico.

- ☐ Regra: chamado nasce ABERTO e dataAbertura é preenchida automaticamente.
- ☐ Criar CRUD base em /chamados.
- ☐ Testar persistência e relacionamentos no H2.
- ☐ Validar com o professor antes do commit.

13.4 Regras de negócio e rotas de ação

- ☐ Criar PATCH /chamados/{id}/atribuir.
- ☐ Criar PATCH /chamados/{id}/status.
- ☐ Impedir atribuição de técnico quando o chamado estiver fechado.
- ☐ Impedir alteração de chamado fechado.
- ☐ Ao fechar, preencher dataFechamento.
- ☐ Testar transições válidas e inválidas.
- ☐ Criar exceção específica se necessário e registrar no GlobalExceptionHandler.
- ☐ Validar com o professor antes do commit.

13.5 Filtros e paginação

- ☐ Implementar GET /chamados?status=&prioridade=&clienteId=.
- ☐ Implementar GET /clientes/{id}/chamados.
- ☐ Adicionar Pageable e ordenação.
- ☐ Testar combinações de filtros.
- ☐ Documentar exemplos de consulta.
- ☐ Validar com o professor antes do commit.

13.6 Comentários e histórico

- ☐ Criar entity Comentario/Interacao com texto, data, autor e chamado.
- ☐ Mapear relação com Chamado.
- ☐ Criar POST /chamados/{id}/comentarios.
- ☐ Listar comentários no detalhe do chamado.

- ☐ Opcional: criar histórico de mudanças de status.
- ☐ Validar com o professor antes do commit.

13.7 Testes automatizados

- ☐ Criar testes de Repository com @DataJpaTest.
- ☐ Criar testes de Controller com @WebMvcTest.
- ☐ Cobrir criação, validação, duplicidade e 404.
- ☐ Garantir que os testes rodem pelo Maven.
- ☐ Validar com o professor antes do commit.

13.8 Fases finais opcionais

- ☐ Adicionar autenticação e autorização com Spring Security.
- ☐ Criar dashboard por status/prioridade.
- ☐ Documentar API com Swagger/OpenAPI.
- ☐ Migrar para PostgreSQL utilizando profiles.
- ☐ Criar dados iniciais para demonstração.
- ☐ Planejar implementação do front-end conforme o mapa de telas.

14 PADRÃO DE ENTREGA DE CADA AULA

Toda aula deve terminar com uma evidência concreta. O aluno deve manter no repositório a implementação correspondente e, quando solicitado, registrar no README a rota criada e um exemplo de teste.

Antes do commit	Durante o commit	Depois do commit
Executar aplicação e testes; revisar git diff; chamar o professor e obter validação.	Usar mensagem objetiva; incluir somente arquivos da etapa; não commitar arquivos desnecessários.	Push da branch; conferir GitHub; atualizar/abrir PR quando orientado; aguardar revisão.

Nunca commitar antes da validação: Se o professor ainda não validou a etapa, o aluno deve manter as alterações locais e solicitar revisão. **Esta regra vale para todas as entregas.**

15 PERGUNTAS DE VERIFICAÇÃO DO APRENDIZADO

1. Por que o Controller não deve consultar o Repository diretamente?
2. Qual é a diferença entre Entity e DTO?
3. Por que a regra de e-mail duplicado está no Service?
4. Qual é a diferença prática entre HTTP 400, 404, 409 e 500?
5. O que o @Transactional garante no fluxo de atualização?
6. Por que o pacote da classe ChamadosApplication fica acima das demais camadas?
7. Em que situação uma mudança de requisito afetaria o DTO sem necessariamente alterar a Entity?
8. Qual a vantagem de criar uma rota PATCH para atribuição/status em vez de usar somente PUT?
9. Como o DER se relaciona com as anotações @ManyToOne e @OneToMany?
10. Como os diagramas ajudam antes da implementação do front-end?

16 ERROS COMUNS E COMO INVESTIGAR

Sintoma	Causa provável	Ação
Porta 8080 em uso	Outra aplicação está rodando	Encerrar o processo anterior ou configurar outra porta.
404 em /	Nenhuma rota para a raiz	Comportamento esperado; testar /clientes.
404 em /clientes	Controller fora do package scan ou aplicação antiga	Conferir package, anotação @RestController e reiniciar.
405 Method Not Allowed	Método HTTP diferente do mapeado	Conferir GET/POST/PUT/DELETE.
400 Bad Request	JSON inválido ou Bean Validation falhou	Ler detalhes retornados pelo handler.
409 Conflict	Regra de negócio violada	Verificar e-mail duplicado ou regra de status.
500 Internal Server Error	Exceção não tratada/erro de configuração	Ler stack trace no terminal; não adivinhar.
H2 não conecta	JDBC URL diferente da configurada	Usar jdbc:h2:mem:chamadosdb.
mvnw sem permissão no macOS	Arquivo não executável	Executar chmod +x mvnw e tentar novamente.
PowerShell não encontra mvnw	Terminal fora da pasta do projeto	Entrar na pasta onde estão mvnw.cmd e pom.xml.

17 REFERÊNCIAS E RECURSOS

SPRING. Spring Initializr. Disponível em: https://start.spring.io/.

SPRING. Spring Boot Reference Documentation. Disponível em: https://docs.spring.io/spring-boot/.

SPRING. Spring Data JPA. Disponível em: https://spring.io/projects/spring-data-jpa.

GITHUB. GitHub Docs. Disponível em: https://docs.github.com/.

FIELDING, Roy T. Architectural Styles and the Design of Network-based Software Architectures. 2000.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.

ANEXO A - CHECKLIST FINAL DO ALUNO

- ☐ Projeto executa em Java 21 com Maven Wrapper.
- ☐ Arquitetura em camadas organizada.
- ☐ Fluxo Cliente concluído e compreendido.
- ☐ Todas as rotas foram testadas.
- ☐ Erros 400, 404 e 409 foram testados.
- ☐ Banco H2 foi inspecionado.
- ☐ Diagrama e DER foram relacionados ao código.
- ☐ Repositorio GitHub contém Prof Sestito/Profesestito como colaborador.
- ☐ Branch segue feature/nome_aluno-developer.
- ☐ Professor validou cada etapa antes do commit.
- ☐ PR foi criado com revisão do Prof Sestito.
- ☐ Nenhuma ferramenta de IA foi utilizada na etapa proibida.